



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a):

Asignatura:

Grupo:

No de práctica(s):

Integrante(s):

No de lista o brigada:

Semestre:

Fecha de entrega:

Observaciones:

Calificación:

Índice

1. Introducción	2
1.1. Planteamiento del Problema	2
1.2. Motivación	2
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. Marco Teórico	3
3.1. Bibliotecas del Lenguaje y Uso del Paquete java.util	3
3.2. Wrappers y Conversión de Tipos Primitivos	3
3.3. Estructuras de Datos Complejas: Diccionarios y Conjuntos Ordenados (HashMap y TreeSet) . .	3
4. Desarrollo	4
4.1. Actividad 1: Consulta a Inteligencia Artificial Generativa sobre HashMap y TreeSet	4
4.2. Actividad 2: Implementación en Java del Verificador de Números Palíndromos de 5 Dígitos . . .	4
4.3. Casos de Prueba de Entrada y Salida	6
5. Resultados	6
5.1. Informe del Contenido Generado por la IA	6
5.2. Evidencias de Ejecución en Terminal	7
6. Conclusiones	8
7. Referencias	10

1 Introducción

1.1 Planteamiento del Problema

Al desarrollar programas más avanzados en Java, nos damos cuenta de que no es eficiente rehacer funcionalidades que el propio lenguaje ya resuelve mediante sus librerías estándar. En esta tercera práctica abordamos el uso de utilerías y clases generales mediante dos actividades principales que pusimos a prueba en el laboratorio:

1. **Investigación mediante Inteligencia Artificial Generativa:** Cada integrante de nuestra brigada utilizó una IA generativa distinta para formularle un prompt técnico. El propósito fue pedirle que nos explicara cómo funcionan por dentro dos colecciones de Java muy usadas: `HashMap` y `TreeSet`, además de solicitarle tres ejemplos comunes de aplicación en el mundo real para cada una.
2. **Programa Verificador de Números Palíndromos:** Diseñamos y programamos una aplicación en consola que le pide al usuario un número entero de 5 dígitos y determina si se lee igual de izquierda a derecha que de derecha a izquierda. La práctica nos exigió validar que, si el usuario ingresa un número que no sea de 5 dígitos, el programa le muestre un mensaje de error y le vuelva a pedir el valor. Además, tuvimos que atender a dos observaciones importantes del profesor: resolver el cálculo usando al menos un método estático y asegurarnos de que el programa no trabaje con números negativos.

1.2 Motivación

Para nosotros como estudiantes de ingeniería, comprender cómo trabajar con objetos y con las herramientas que ya vienen en el lenguaje es un paso fundamental antes de construir sistemas más complejos. Aprender a usar clases como `Scanner` o `Math`, y entender la diferencia entre tipos primitivos y wrappers, nos ayuda a escribir un código más limpio y sin errores al convertir datos. Por otro lado, saber cómo funcionan estructuras como `HashMap` y `TreeSet` nos da el criterio necesario para elegir la mejor opción según lo que necesitemos guardar o buscar en memoria, y finalmente, aplicar métodos estáticos nos enseña a estructurar nuestro código de forma modular, reutilizando funciones de cálculo sin necesidad de crear objetos innecesarios.

2 Objetivos

2.1 Objetivo General

Utilizar bibliotecas propias del lenguaje para realizar algunas tareas comunes y recurrentes.

2.2 Objetivos Específicos

- Comprender la diferencia entre los tipos de datos primitivos y los wrappers en Java, aplicando métodos para la conversión y validación de datos.
- Implementar métodos estáticos para separar la lógica de cálculo de la interfaz de consola, logrando un código modular e independiente.
- Analizar la estructura interna y los casos de uso de `HashMap` y `TreeSet` apoyándonos en consultas a herramientas de Inteligencia Artificial Generativa.

3 Marco Teórico

3.1 Bibliotecas del Lenguaje y Uso del Paquete `java.util`

Java cuenta con una amplia variedad de clases predefinidas organizadas en paquetes dentro de su API estándar. El paquete `java.lang` se importa de manera automática y contiene las herramientas más básicas como `System`, `Math` y `String`. Por otro lado, cuando necesitamos interactuar con el usuario o manipular colecciones de datos, recurrimos al paquete `java.util`, donde encontramos herramientas como la clase `Scanner` para la lectura de datos por teclado y las colecciones dinámicas para almacenar objetos [1].

3.2 Wrappers y Conversión de Tipos Primitivos

Los tipos de datos primitivos como `int` o `double` guardan directamente su valor en la memoria rápida (stack), pero no tienen métodos. Para trabajar con ellos como si fueran objetos, Java ofrece las clases envoltoras o wrappers (`Integer`, `Double`, `Character`, etc.) [2].

Estas clases son muy útiles en nuestros programas porque nos proporcionan métodos estáticos como `Integer.parseInt()`, los cuales nos permiten convertir textos ingresados por el usuario a valores numéricos de forma sencilla [2, 3]. De igual forma, el lenguaje realiza de manera automática los procesos de autoboxing (convertir de primitivo a objeto) y unboxing (extraer el valor primitivo del objeto) [2].

Por otra parte, los métodos estáticos (`static`) pertenecen a la clase como tal y no a un objeto en específico [3]. Esto nos permite invocarlos directamente sin tener que usar el operador `new`, lo que resulta ideal para crear funciones de validación o cálculos matemáticos [3].

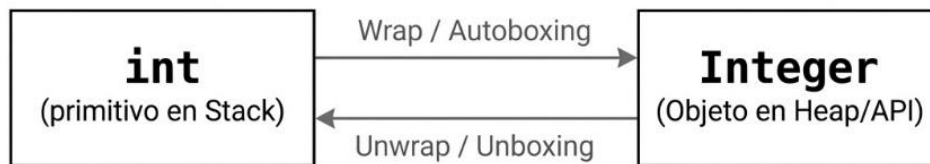


Figura 1. Conversión entre un tipo primitivo `int` y su clase envoltora `Integer` mediante wrapping/autoboxing y unwrapping/unboxing.

3.3 Estructuras de Datos Complejas: Diccionarios y Conjuntos Ordenados (`HashMap` y `TreeSet`)

Dentro del marco de colecciones de Java (`java.util`), existen estructuras diseñadas para organizar la información de forma eficiente según las necesidades del programa:

- **HashMap:** Es una estructura que almacena la información en parejas de clave y valor (Key-Value) [4]. Utiliza un mecanismo llamado hashing que convierte la clave en una posición de memoria, lo que permite buscar, agregar o borrar elementos de forma prácticamente inmediata sin importar el tamaño de la lista [4]. No guarda ningún orden entre sus elementos [4].
- **TreeSet:** Es una colección que almacena elementos únicos (no permite duplicados) y los mantiene ordenados automáticamente de forma ascendente [5]. Por dentro utiliza un árbol binario balanceado (árbol rojo-negro), lo que hace que las búsquedas e inserciones se realicen de manera rápida y estructurada [5].

4 Desarrollo

4.1 Actividad 1: Consulta a Inteligencia Artificial Generativa sobre HashMap y TreeSet

Para realizar esta actividad, cada uno de los 5 integrantes de nuestra brigada utilizó una herramienta de Inteligencia Artificial Generativa distinta. Para asegurarnos de obtener una respuesta clara y con buen nivel técnico, redactamos un prompt común bien estructurado:

Prompt elaborado por la brigada:

“Actúa como un profesor de programación en Java. Explica de manera clara y directa cómo funcionan por dentro las clases HashMap y TreeSet del paquete java.util. Además, proporciona 3 ejemplos comunes y reales de aplicación para cada una de estas dos estructuras.”

Al recopilar y comparar las respuestas obtenidas por cada integrante, armamos el siguiente esquema para resumir las diferencias principales entre ambas colecciones:

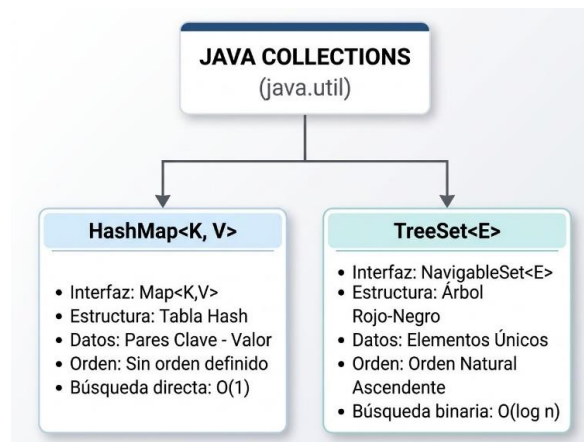


Figura 2. Esquema comparativo de las colecciones HashMap<K, V> y TreeSet<E> del paquete java.util.

4.2 Actividad 2: Implementación en Java del Verificador de Números Palíndromos de 5 Dígitos

Para resolver el Ejercicio 2, escribimos el programa en el archivo Palindromo5Digitos.java. Decidimos separar el programa en dos partes: el manejo de la consola en el método main y la comprobación del palíndromo dentro de un método estático.

Funcionamiento del Método Estático esPalindromo(int numero)

En lugar de convertir el número a texto, aprovechamos las operaciones matemáticas de división entera / y módulo % para separar dígito por dígito. Esto hace que el cálculo sea más directo y rápido:

1. Obtener el primer y quinto dígito:

- **Primer dígito:** numero / 10000. Al dividir un número de 5 dígitos entre 10000 nos queda únicamente el primer número.
- **Quinto dígito:** numero % 10. El módulo 10 nos da el residuo, que es el último dígito.

2. Obtener el segundo y cuarto dígito:

- **Segundo dígito:** $(\text{numero} / 1000) \% 10$.
 - **Cuarto dígito:** $(\text{numero} / 10) \% 10$.
3. **Comparación:** El método regresa un valor verdadero (`true`) únicamente si el primer dígito es igual al quinto y el segundo dígito es igual al cuarto.

Lógica del Programa Principal (main) y Validación

1. **Lectura de datos:** Usamos la clase Scanner para pedir el número. Para cumplir con la indicación del profesor de no permitir números negativos, aplicamos el método estático `Math.abs()` de la librería `java.lang`, convirtiendo cualquier número negativo a positivo automáticamente.
2. **Validación de 5 dígitos con do-while:** Revisamos que el valor esté dentro del rango permitido ($10000 \leq \text{numero} \leq 99999$). Si el número tiene menos o más de 5 dígitos, el programa imprime “Error: el número debe tener 5 dígitos” y vuelve a solicitar el dato mediante el ciclo `do-while`.
3. **Impresión del resultado:** Cuando el usuario ingresa un número válido de 5 dígitos, llamamos a nuestro método estático `esPalindromo(numero)` y mostramos en pantalla si es o no un palíndromo.

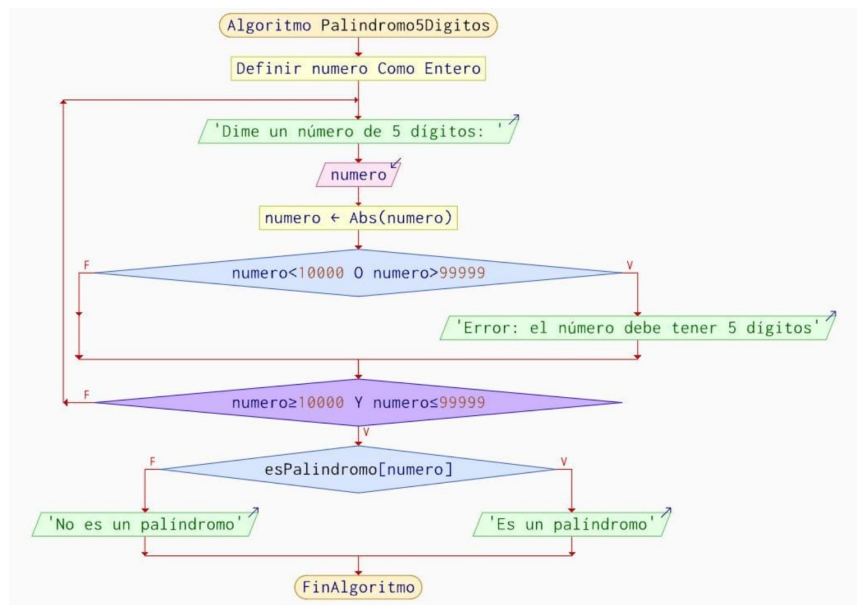


Figura 3. Diagrama de flujo del programa principal de Palindromo5Digitos, que valida la entrada y realiza la llamada al método estático `esPalindromo`.

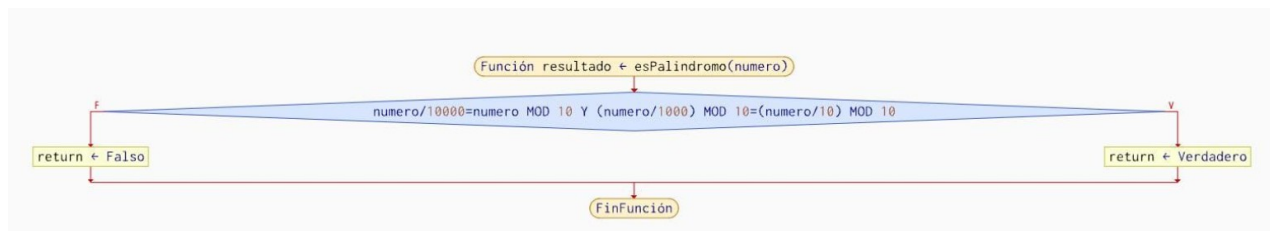


Figura 4. Diagrama de flujo del método estático `esPalindromo`, encargado de comparar los dígitos extremos e internos del número.

Los dos diagramas corresponden al mismo archivo de clase: el flujo principal utiliza la función `esPalindromo(numero)` como llamada a un método estático para realizar la comprobación.

4.3 Casos de Prueba de Entrada y Salida

Llevamos a cabo diversas pruebas para comprobar que nuestro programa funcionara correctamente ante distintas entradas. Los valores se ajustaron para que coincidieran con las ejecuciones mostradas posteriormente en las evidencias de terminal:

1. **Prueba 1: Número de 5 dígitos no palíndromo:**

- **Entrada:** 10000.
- **Resultado:** El programa reconoce que tiene 5 dígitos y determina que no es un palíndromo.

2. **Prueba 2: Validación de entrada y número no palíndromo:**

- **Entrada inicial:** 00500. El programa muestra “Error: el número debe tener 5 dígitos”.
- **Segunda entrada:** 11522. Pasa la validación de rango, pero no cumple la condición de palíndromo, por lo que imprime “No es un palíndromo”.

3. **Prueba 3: Palíndromos con diferentes patrones:**

- **Entrada:** 22522. El método compara los dígitos y determina que es un palíndromo.
- **Entrada:** 99999. Pasa la validación e imprime “Es un palíndromo”.

4. **Prueba 4: Casos adicionales y control de errores:**

- **Entrada:** 10001. El programa determina que es un palíndromo.
- **Entrada:** 00000. El valor es menor que 10000 y se muestra el mensaje de error.
- **Reintento:** 90000. Pasa la validación de 5 dígitos y se determina que no es un palíndromo.

5 Resultados

5.1 Informe del Contenido Generado por la IA

A partir del protocolo de consulta ejecutado por la brigada, analizamos e integramos la información teórica y práctica que nos entregó la Inteligencia Artificial al responder sobre la arquitectura de ambas colecciones:

1. Arquitectura y Funcionamiento Interno de HashMap

La IA nos detalló que `HashMap` pertenece a la interfaz `Map` y organiza la información en parejas de clave y valor mediante una tabla de dispersión (hash table):

- **Mecanismo de Hashing:** Cuando guardamos un dato con su clave, Java ejecuta el método `hashCode()` sobre la clave para generar un número entero. Este número se procesa matemáticamente para determinar el índice o casilla (bucket) exacta dentro del arreglo interno donde se almacenará el objeto.
- **Manejo de Colisiones:** Si dos claves distintas generan el mismo índice, ocurre una colisión. La IA nos

explicó que Java resuelve esto guardando los elementos en una lista enlazada dentro de esa misma casilla. Si se acumulan más de 8 elementos en la misma posición, Java transforma esa lista en un árbol rojo-negro para evitar que la búsqueda se vuelva lenta.

- **Contrato hashCode() y equals():** Para buscar una clave, Java primero localiza la casilla con hashCode() y después usa el método .equals() para asegurarse de que sea exactamente la clave que estamos pidiendo.
- **Complejidad Temporal:** Gracias a la dispersión en casillas, las operaciones de búsqueda, inserción y eliminación son casi inmediatas, logrando un tiempo constante promedio $O(1)$. Cabe destacar que no guarda ningún tipo de orden entre los elementos.

3 aplicaciones comunes de HashMap:

- **Catálogos de productos o inventarios:** Asociar el código único o código de barras del producto como clave y el objeto con sus detalles (nombre, precio, stock) como valor.
- **Gestión de sesiones de usuario:** Guardar en un servidor web la sesión activa de cada usuario utilizando un token de autenticación como clave de acceso.
- **Conteo de frecuencia de elementos:** Registrar cuántas veces se repite cada palabra dentro de un texto, usando la palabra como clave y el contador como valor.

2. Arquitectura y Funcionamiento Interno de TreeSet

Respecto a TreeSet, la IA nos explicó que es una estructura perteneciente a la interfaz Set que garantiza dos características principales: no permite elementos repetidos y los mantiene ordenados automáticamente.

- **Estructura de Árbol Rojo-Negro (Red-Black Tree):** Por dentro, TreeSet organiza los datos utilizando un árbol binario de búsqueda auto-balanceado. Cada elemento representa un nodo con ramificaciones hacia la izquierda y derecha.
- **Criterio de Ordenamiento:** Al insertar un nuevo dato, el árbol lo compara contra los elementos existentes utilizando su orden natural (mediante la interfaz Comparable, que ya viene en clases como Integer o String) o usando un comparador personalizado. Si al comparar encuentra que el elemento ya existe, simplemente lo descarta.
- **Complejidad Temporal:** Como la estructura de árbol se mantiene balanceada en todo momento, las operaciones de búsqueda, inserción y eliminación tienen una complejidad logarítmica $O(\log n)$.

3 aplicaciones comunes de TreeSet:

- **Tabla de posiciones en videojuegos (Leaderboards):** Almacenar los puntajes de los jugadores garantizando que no existan valores duplicados y manteniéndolos ordenados de mayor a menor de forma automática.
- **Agendas y programación de eventos:** Organizar citas o tareas por fecha u hora, asegurando que se muestren en orden cronológico conforme se agregan.
- **Diccionarios para autocompletado:** Guardar palabras o términos de búsqueda de manera alfabética y sin repeticiones para realizar búsquedas por prefijo de forma rápida.

5.2 Evidencias de Ejecución en Terminal

Capturas de Pantalla de la Terminal


```

Austria12:Downloads poo07alu08$ java Palindromo5Digitos
Error: el número debe tener 5 dígitos
Dime un número de 5 dígitos: 10000
No es un palíndromo
● Austria12:Downloads poo07alu08$ java Palindromo5Digitos
Dime un número de 5 dígitos: 00500

```

Figura 5. Ejecución con la entrada 10000 y verificación de que el número, aunque tiene cinco dígitos, no es un palíndromo.

```

Dime un número de 5 dígitos: 00500
Error: el número debe tener 5 dígitos
Dime un número de 5 dígitos: 11522
No es un palíndromo
● Austria12:Downloads poo07alu08$ java Palindromo5Digitos
Dime un número de 5 dígitos: 22522
Es un palíndromo
● Austria12:Downloads poo07alu08$ java Palindromo5Digitos
Dime un número de 5 dígitos: 99999

```

Figura 6. Validación de una entrada con formato incorrecto (00500) y posterior prueba con 11522, que no es un palíndromo.

```

Dime un número de 5 dígitos: 99999
Es un palíndromo
● Austria12:Downloads poo07alu08$ javac Palindromo5Digitos.java
● Austria12:Downloads poo07alu08$ java Palindromo5Digitos
Dime un número de 5 dígitos: 10001
Es un palíndromo
● Austria12:Downloads poo07alu08$ java Palindromo5Digitos
Dime un número de 5 dígitos: 00000
Error: el número debe tener 5 dígitos
Dime un número de 5 dígitos: 90000
No es un palíndromo
○ Austria12:Downloads poo07alu08$ 

```

Figura 7. Ejecuciones adicionales que verifican palíndromos (22522, 99999 y 10001) y el control de entradas fuera del rango permitido (00000 y 90000).

6 Conclusiones

Al finalizar esta tercera práctica de laboratorio, logramos cumplir satisfactoriamente con todos los objetivos planteados. A lo largo de las actividades comprobamos lo importante que es apoyarse en las librerías estándar de Java para realizar tareas comunes como la lectura de datos, la conversión de tipos y el manejo de valores absolutos sin tener que reinventar código. Aprendimos a estructurar nuestros programas de forma más limpia dividiendo las tareas en métodos estáticos, lo que nos permitió aislar la lógica de comprobación matemática del palíndromo y reutilizarla fácilmente dentro del flujo principal del programa. Asimismo, la investigación realizada sobre las colecciones del paquete `java.util` nos ayudó a entender cuándo nos conviene utilizar una estructura

de acceso rápido como `HashMap` y cuándo es mejor optar por el ordenamiento automático que ofrece `TreeSet`. En conjunto, la práctica nos permitió reforzar la forma en que manejamos las utilerías del lenguaje y nos dio una base sólida para seguir construyendo aplicaciones mejor estructuradas en las siguientes sesiones.

7 Referencias

Referencias

- [1] M. Sierra y J. L. Cuenca, “Paquete java.util (API Java). Interfaces y clases: StringTokenizer, Date, Calendar, HashSet, TreeSet (CU00916C),” *aprenderaprogramar.com*, 2006. [En línea]. Disponible en: https://www.aprenderaprogramar.com/index.php?option=com_content&view=article&id=595:paquete-javaut-il-api-java-interfaces-y-clases-stringtokenizer-date-calendar-hashset-treeset-cu00916c&catid=58&Itemid=180.
- [2] V. Tech, “Primitive vs Wrapper classes in Java,” *Medium*, 3 de oct. de 2023. [En línea]. Disponible en: <https://medium.com/@vino7tech/primitive-vs-wrapper-classes-in-java-81c50c33f30a>.
- [3] Coddy, “Métodos estáticos en Java,” *Coddy*, 2024. [En línea]. Disponible en: https://coddy.tech/learn/es/java/object_oriented_programming/static_methods.
- [4] W3Schools, “Java HashMap,” *W3Schools Online Web Tutorials*, 2024. [En línea]. Disponible en: https://www.w3schools.com/java/java_hashmap.asp.
- [5] GeeksforGeeks, “TreeSet in Java with Examples,” *GeeksforGeeks Computer Science Portal*, 2024. [En línea]. Disponible en: <https://www.geeksforgeeks.org/java/treeset-in-java-with-examples/>.